Vaultwarden Security Audit Report

Part 1: Executive Summary & Threat Model

Audit Date: January 2026 **Application:** Vaultwarden v1.0.0 **Auditor:** Security Assessment Team **Repository:** https://github.com/dani-garcia/vaultwarden

---

Executive Summary

Overview

Vaultwarden is an alternative server implementation of the Bitwarden Client API, written in Rust. It provides a self-hosted password management solution compatible with official Bitwarden clients. This security audit evaluated the application's architecture, authentication mechanisms, cryptographic implementations, input validation, and overall security posture.

Key Findings Summary

Overall Security Posture: GOOD with areas for improvement

The application demonstrates strong security practices in several critical areas: - ✅ **Excellent cryptographic foundation** using industry-standard libraries (ring, argon2) - ✅ **Strong SSRF protection** with comprehensive IP blocking and DNS validation - ✅ **Proper ORM usage** (Diesel) preventing SQL injection - ✅ **Zero-knowledge architecture** - server never sees plaintext vault data - ✅ **Comprehensive authentication** with JWT, SSO/OIDC, WebAuthn/FIDO2, and multiple 2FA options - ✅ **Forbidden unsafe code** - Rust's memory safety guarantees enforced - ⚠️ **Some configuration security concerns** requiring attention - ⚠️ **Complex security stamp exception logic** with potential edge cases - ⚠️ **Limited rate limiting** on certain endpoints

Critical Findings: 0 **High Severity:** 0 **Medium Severity:** 3 **Low Severity:** 5 **Informational:** 8

Scope of Assessment

This audit covered: - Authentication and authorization mechanisms - Cryptographic implementations - Input validation and sanitization - SQL injection and command injection vectors - Cross-Site Scripting (XSS) vulnerabilities - Server-Side Request Forgery (SSRF) -

Sensitive data exposure - Configuration security - Session management - Audit logging capabilities - Dependency vulnerabilities - STRIDE threat modeling

---

Threat Model

Application Architecture

Vaultwarden is a Rust-based web application built on the Rocket framework that implements:

Core Components: - **Web API** (Rocket 0.5.1) - REST API endpoints - **Database Layer** (Diesel ORM) - Supports SQLite, MySQL, PostgreSQL - **Authentication System** - JWT with RS256, SSO/OIDC, 2FA - **Cryptographic Engine** - Client-side encryption, server-side password hashing - **WebSocket Service** - Real-time notifications - **Icon Service** - Favicon fetching with SSRF protection - **Email Service** - SMTP integration for notifications - **Admin Panel** - Web-based administration interface - **Storage Backend** - Local filesystem or S3-compatible storage (via OpenDAL)

Trust Boundaries

Trust Boundary 1: External Network ↔ Application

Crossing Point: HTTP/HTTPS requests from clients **Trust Level Change:** Untrusted → Application Context **Security Controls:** - TLS/HTTPS encryption (recommended via reverse proxy) - Rate limiting on login endpoints - JWT token validation - Origin validation - CSRF protection via SameSite cookies

Trust Boundary 2: Application ↔ Database

Crossing Point: SQL queries via Diesel ORM **Trust Level Change:** Application Context → Data Layer **Security Controls:** - Parameterized queries via Diesel ORM - Connection pooling with R2D2 - Database-level access controls - Foreign key constraints

Trust Boundary 3: Application ↔ External Services

Crossing Point: Outbound HTTP requests **Trust Level Change:** Application Context → External Network **Security Controls:** - Custom DNS resolver with IP blocking - Non-global IP address filtering (RFC 1918, link-local, etc.) - Regex-based domain blocking - Maximum 5 redirect limit - 10-second timeout - Redirect host validation

Trust Boundary 4: Authenticated User ↔ Organization Resources

Crossing Point: Organization/collection access checks **Trust Level Change:** User Context → Organization Context **Security Controls:** - Role-Based Access Control (Owner/Admin/Manager/User) - Collection-level permissions - Membership status validation (Invited/Accepted/Confirmed) - Security stamp validation

Trust Boundary 5: Admin ↔ Admin Panel

Crossing Point: Admin panel authentication **Trust Level Change:** User Context → Admin Context **Security Controls:** - Argon2id hashed ADMIN_TOKEN - Session-based authentication with JWT - Configurable session lifetime (default: 20 minutes) - Rate limiting on admin login - HTTP-only cookies with SameSite=Strict

Trust Boundary 6: Application ↔ File Storage

Crossing Point: File operations (attachments, icons, backups) **Trust Level Change:** Application Context → Storage Layer **Security Controls:** - OpenDAL abstraction layer - Configurable storage backends (filesystem, S3) - File path validation - Size limits on uploads - Content-type validation

Data Flow Analysis

Critical Data Flows

1. User Registration & Login Flow

```
Client → [POST /identity/accounts/register]
  ├─> Email validation
  ├─> Domain whitelist check (if enabled)
  ├─> Password policy enforcement
  ├─> Generate 64-byte salt
  ├─> Hash password with PBKDF2 (600K iterations) or Argon2id
  ├─> Generate security stamp (UUID)
  ├─> Store in database
  └─> Return success

Client → [POST /identity/connect/token] (grant_type=password)
  ├─> Rate limit check (LOGIN_RATELIMIT)
  ├─> Retrieve user by email
  ├─> Verify password hash (constant-time comparison)
  ├─> Check 2FA requirements
  ├─> Generate device record
  ├─> Create JWT access token (RS256, 2-hour expiry)
  ├─> Create refresh token (30-90 days based on device)
  ├─> Log event (EventType::UserLoggedIn)
  └─> Return tokens
```

2. SSO/OIDC Authentication Flow

```
Client → [GET /identity/connect/authorize]
  ├─> Generate PKCE code_verifier (128 bytes random)
  ├─> Generate code_challenge (SHA256 hash)
  ├─> Generate state parameter (anti-CSRF)
  ├─> Store in sso_auth table (5-minute expiry)
  ├─> Redirect to IdP authorization endpoint
  └─> Return redirect URL

IdP → [GET /identity/connect/oidc-signin] (callback)
  ├─> Validate state parameter
  ├─> Validate code_verifier via PKCE
  ├─> Exchange code for tokens
  ├─> Validate ID token signature
  ├─> Extract claims (identifier, email, email_verified)
  ├─> Match or create user
  ├─> Handle 2FA if required
  ├─> Generate device and tokens
  └─> Return authentication response
```

3. Vault Sync Data Flow

```
Client → [GET /api/sync] (with Bearer token)
  ├─> Decode JWT access token
  ├─> Validate security stamp
  ├─> Check stamp exception (2-minute window for multi-step ops)
  ├─> Retrieve user's ciphers (encrypted)
  ├─> Retrieve folders
  ├─> Retrieve collections
  ├─> Retrieve organization data
  ├─> Retrieve sends
  ├─> Retrieve policies
  └─> Return encrypted JSON (server never sees plaintext)
```

4. Cipher Creation/Update Flow

```
Client → [POST /api/ciphers]
  ├─> Authenticate via Headers guard
  ├─> Validate JSON structure
  ├─> Check organization membership (if org cipher)
  ├─> Verify collection access
  ├─> Validate cipher type (1=Login, 2=Note, 3=Card, 4=Identity)
  ├─> Store encrypted data (server-side encrypted fields)
  ├─> Update user's updated_at timestamp
  ├─> Log event (EventType::CipherCreated)
  ├─> Trigger WebSocket notification
  └─> Return cipher JSON
```

5. Attachment Upload Flow

```
Client → [POST /api/ciphers/{id}/attachment/v2]
  ├─> Authenticate user
  ├─> Verify cipher ownership
  ├─> Validate attachment size
  ├─> Generate attachment ID (80 bits random)
  ├─> Generate file download token (JWT, 5-minute expiry)
  ├─> Store metadata in database
  └─> Return upload URL

Client → [POST /api/ciphers/{id}/attachment/{attachment_id}] (multipart upload)
  ├─> Validate file download token
  ├─> Write to storage (filesystem or S3)
  ├─> Update attachment size in DB
  ├─> Log event (EventType::CipherAttachmentCreated)
  └─> Return success
```

6. Icon Fetching Flow (SSRF Risk Mitigation)

```
Client → [GET /icons/{domain}/icon.png]
  ├─> Validate domain format
  ├─> Check icon cache
  ├─> DNS resolution with custom resolver
  │   ├─> Block non-global IPs (RFC 1918, link-local, loopback)
  │   ├─> Apply regex blocking rules
  │   └─> Validate resolved IP is global
  ├─> HTTP request with redirect validation
  │   ├─> Max 5 redirects
  │   ├─> Validate each redirect host
  │   ├─> 10-second timeout
  │   └─> Custom user-agent
  ├─> Parse HTML for icon links
  ├─> Download and sanitize SVG (if applicable)
  ├─> Cache icon (configurable TTL)
  └─> Return icon or fallback
```

7. Admin Panel Access Flow

```
Client → [POST /admin] (login form)
  ├─> Rate limit check (ADMIN_RATELIMIT)
  ├─> Validate ADMIN_TOKEN
  │   ├─> If Argon2id PHC: verify hash
  │   ├─> Else: constant-time string comparison
  │   └─> Reject on failure
  ├─> Generate admin JWT (20-minute expiry)
  ├─> Set HTTP-only cookie with SameSite=Strict
  └─> Return admin panel

Admin → [Any admin endpoint]
  ├─> Validate admin JWT from cookie
  ├─> Decode and verify signature
  ├─> Check expiration
  └─> Allow access
```

STRIDE Threat Analysis

S - Spoofing Identity

Threat	Impact	Likelihood	Current Controls	Risk Level
JWT Token Theft	Attacker gains full access to user account	Medium	- HTTP-only cookies - Secure flag on HTTPS - Short expiration (2 hours) - Security stamp validation	**Medium**
Session Fixation	Attacker forces user to use known session	Low	- Tokens generated server-side - Cryptographically random - No session ID in URL	**Low**
SSO Token Manipulation	Attacker bypasses authentication	Low	- PKCE with code_verifier - State parameter validation - Token signature verification - 5-minute token expiry	**Low**
Admin Token Brute Force	Attacker gains admin access	Medium	- Argon2id hashing recommended - Rate limiting - No account lockout (concern)	**Medium**
2FA Bypass	Attacker skips second factor	Low	- Device binding - Time-based codes - One-time use enforcement - Remember-me limited to 30 days	**Low**

Recommendations: - ⚠️ Enforce Argon2id ADMIN_TOKEN (deprecate plaintext) - ✅ Consider account lockout after N failed admin attempts - ✅ Implement device fingerprinting for anomaly detection

---

T - Tampering with Data

Threat	Impact	Likelihood	Current Controls	Risk Level
Cipher Data Tampering	Attacker modifies vault items	Low	- Client-side encryption (zero-knowledge) - Server only stores encrypted data - Integrity via HMAC in client	**Low**
SQL Injection	Database compromise	Very Low	- Diesel ORM with parameterized queries - No raw SQL found in user input paths - Type-safe query builder	**Very Low**
Configuration Tampering	Malicious config changes	Medium	- File permissions (OS-level) - No config validation on runtime changes - Admin panel allows config updates	**Medium**
Organization Data Modification	Unauthorized org changes	Low	- Role-based access controls - Owner/Admin/ Manager hierarchy - Collection permissions	**Low**
Event Log Tampering	Audit trail manipulation	Low	- No delete functionality exposed - Timestamp auto-generated - Retention policy available	**Low**

Recommendations: - ✅ Strong file permissions on config.json and .env - ✅ Consider config file integrity monitoring - ⚠️ Add checksums/signatures for critical config changes

---

R - Repudiation

Threat	Impact	Likelihood	Current Controls	Risk Level
User Denies Actions	Cannot prove user actions	Medium	- Event logging system (EventType enum) - IP address logging - Device type tracking - Timestamp recording	**Low**
Admin Actions Not Logged	Admin abuse without trace	Medium	- Admin actions create events - ACTING_ADMIN_USER constant - Limited to org-scoped events	**Medium**
Failed Login Not Logged	Brute force attempts hidden	Low	- EventType::UserFailedLogIn - EventType::UserFailedLogIn2fa - IP address captured	**Low**
Event Log Gaps	Missing security-relevant events	Medium	- 35+ event types supported - Some client-side events not captured - No centralized audit export	**Medium**

Recommendations: - ⚠️ Enhance admin action logging (user management, config changes) - ✅ Add event log export functionality (CSV/JSON) - ✅ Implement log retention policies - ✅ Consider SIEM integration (syslog support exists)

I - Information Disclosure

Threat	Impact	Likelihood	Current Controls	Risk Level
Error Messages Leak Info	Stack traces expose internals	Low	- Custom error types - Generic user-facing messages - Debug info only in logs	**Low**
Timing Attacks on Login	Username enumeration	Medium	- Constant-time password comparison - Same error message for invalid user/password - Consistent timing (good)	**Low**
Admin Token in Logs	Credentials exposed in logs	Medium	- Masked in support JSON - Not logged in normal operations - Risk if plain-text used	**Medium**
Database Credentials	Connection strings exposed	Medium	- Masked in support output - Environment variables used - Not exposed via API	**Low**
JWT Claims Verbose	Token size/ content disclosure	Low	- Removed org lists from JWT (optimization) - Minimal claims included - Standard JWT structure	**Very Low**
Password Hints	Weak hints aid attacks	High	- Optional feature (configurable) - Stored plaintext - Sent via email	**Medium**

Recommendations: - ⚠️ Disable PASSWORD_HINTS_ALLOWED by default (SOC 2 concern) - ✅ Consider rate limiting password hint requests - ⚠️ Add warning about plain-text ADMIN_TOKEN in startup logs

D - Denial of Service

Threat	Impact	Likelihood	Current Controls	Risk Level
Login Brute Force	Account lockout or resource exhaustion	High	- Rate limiting (LOGIN_RATELIMIT) - Governor crate - Configurable burst/ seconds	**Low**
Admin Brute Force	Admin panel unavailable	Medium	- Admin rate limiting - No account lockout - Session-based access	**Medium**
Icon Service Abuse	Resource exhaustion via icon fetches	Medium	- Domain validation - Caching (TTL-based) - Timeout (10s) - Max redirects (5)	**Low**
WebSocket Flooding	Connection exhaustion	Medium	- No apparent rate limiting - Authentication required - Per-user subscriptions	**Medium**
Large Attachment Upload	Storage/memory exhaustion	High	- Size limits configurable - Streaming uploads - Storage backend abstraction	**Low**
Database Connection Pool	DB connection exhaustion	Medium	- R2D2 connection pooling - Configurable pool size - Timeout settings	**Low**

Recommendations: - ⚠️ Add rate limiting to WebSocket connections - ✅ Implement per-user attachment storage quotas - ✅ Add monitoring for resource exhaustion - ⚠️ Consider admin account lockout after N failures

E - Elevation of Privilege

Threat	Impact	Likelihood	Current Controls	Risk Level
Org User → Admin Escalation	Unauthorized admin access	Low	- Membership type validation - Confirmed status required - Server-side checks on all ops	**Very Low**
User → Org Owner Escalation	Take over organization	Very Low	- Strict role hierarchy - Owner-only operations enforced - No self-promotion paths	**Very Low**
Security Stamp Bypass	Session hijacking after password change	Medium	- Security stamp in JWT - Stamp exception logic (complex) - 2-minute exception window	**Medium**
Collection Access Bypass	Access restricted ciphers	Low	- Manager/Admin checks - Collection membership validation - Read-only flags enforced	**Low**
Admin Panel Access	Gain admin panel control	Medium	- Token-based authentication - Plain-text token accepted (legacy) - No MFA on admin login	**Medium**
SSO Account Takeover	Link existing account via SSO	Low	- Email matching optional - Private key check before linking - SSO_SIGNUPS_MATCH_EMAIL flag	**Low**

Recommendations: - ⚠️ Simplify security stamp exception logic (reduce attack surface) - ⚠️ Enforce Argon2id ADMIN_TOKEN only - ✅ Add MFA option for admin panel - ✅ Audit stamp exception usage for edge cases

---

Attack Surface Analysis

External Attack Surface

1. Public API Endpoints - `/identity/*` - Authentication endpoints (highest risk) - `/api/*` - Core API (authenticated, medium risk) - `/admin` - Admin panel (high risk if token weak) - `/icons/*` - Icon service (SSRF risk - mitigated) - `/notifications/hub` - WebSocket (authenticated, DoS risk)

2. Authentication Mechanisms - Password authentication (brute force risk - mitigated) - SSO/OIDC (misconfiguration risk) - API keys (theft/exposure risk) - 2FA (bypass risk - low) - WebAuthn/FIDO2 (phishing-resistant)

3. External Service Integrations - SMTP servers (credential exposure) - SSO Identity Providers (trust relationship) - Yubico OTP API (network dependency) - Duo Security (API credential exposure) - Icon fetching (SSRF risk - well mitigated)

Internal Attack Surface

1. Database Access - Diesel ORM queries (SQL injection - very low risk) - Connection pooling (DoS risk - low) - Migration system (integrity risk if compromised)

2. File System Operations - Attachment storage (path traversal - checked) - Icon caching (directory traversal - validated) - Config file reading (permission-based) - Database backups (SQLite only)

3. Inter-Component Communication - WebSocket subscriptions (authenticated) - Background job scheduler (internal only) - Push notification relay (authenticated, external dependency)

Compliance Considerations (SOC 2 Relevant)

Access Controls (CC6.1 - CC6.3)

✅ **Strong**: Role-based access control with Owner/Admin/Manager/User hierarchy ✅ **Strong**: Collection-level permissions with read-only flags ⚠️ **Concern**: Admin panel lacks MFA option ⚠️ **Concern**: No account lockout policy for admin accounts

Audit Logging (CC7.2)

✅ **Strong**: Comprehensive event system with 35+ event types ✅ **Strong**: IP address and device type logging ⚠️ **Concern**: Limited admin-specific action logging ⚠️ **Concern**: No automated log export or SIEM integration (syslog available but manual)

Encryption (CC6.1)

✅ **Strong**: Client-side encryption (zero-knowledge architecture) ✅ **Strong**: TLS recommended for transport ✅ **Strong**: Argon2id password hashing ⚠️ **Concern**: Plain-text ADMIN_TOKEN still accepted (legacy support)

Data Retention (CC7.3)

✅ **Strong**: Configurable event retention policy ✅ **Strong**: Soft-delete for ciphers (trashed state) ⚠️ **Informational**: No automated data retention enforcement

Authentication (CC6.1)

✅ **Strong**: Multi-factor authentication support ✅ **Strong**: WebAuthn/FIDO2 phishing-resistant ✅ **Strong**: SSO/OIDC integration ⚠️ **Concern**: Password hints stored in plaintext (optional feature)

Vulnerability Management (CC7.1)

✅ **Strong**: Regular dependency updates ✅ **Strong**: Rust's memory safety guarantees ⚠️ **Concern**: No automated dependency vulnerability scanning in CI/CD (cargo-audit not run)

Summary & Next Steps

Overall Assessment

Vaultwarden demonstrates **strong security fundamentals** with a well-architected zero-knowledge design. The use of Rust provides memory safety guarantees, and the application employs industry-standard cryptographic libraries. The SSRF protection is particularly well-implemented with multiple layers of validation.

Primary Concerns

1. **Configuration Security**: Plain-text ADMIN_TOKEN acceptance should be deprecated

2. **Security Stamp Logic**: Complex exception mechanism needs simplification
3. **Admin Panel**: Lacks MFA and has limited action logging
4. **Password Hints**: Plaintext storage presents information disclosure risk
5. **Rate Limiting**: WebSocket connections lack rate limiting

Strengths

- Excellent cryptographic implementation
- Strong SSRF mitigation
- Comprehensive authentication options
- Zero-knowledge architecture
- Memory-safe implementation (Rust)
- Good use of ORM preventing SQL injection

End of Report 1: Executive Summary & Threat Model

Next Report: Critical & High Severity Findings (detailed vulnerabilities with exploitation scenarios)